Representation Determinism vs. Model-Output Determinism: The Standalone Architectural Distinction Between What CKS Requires Deterministic and What It Does Not

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 5, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the **substrate/model determinism distinction** — the architectural boundary between the determinism CKS requires of substrate representation and the determinism CKS explicitly does not require of LLM model-output — as a standalone architectural commitment with independent operational content, separable from the five guarantees, the allowed non-determinism categories, and the regression-testing application that elaborate the determinism contract integratively.

Abstract

The CKS determinism contract commits a substrate to specific reproducibility properties while operating with LLM components whose outputs vary across executions. The contract's coherence depends on a precise architectural boundary between two kinds of determinism: **representation determinism** at the substrate layer (a property of substrate content, persistence, and read behavior) and **model-output determinism** at the LLM layer (a property of LLM token outputs across identical-input executions). The integrating-frame note for the contract decomposition (A2.55) names this boundary at the contract level. This note formalizes the boundary as having independent architectural content — the distinction is what permits the contract to operate with current LLM technology while preserving substrate's commitments. The note states what each form of determinism means (representation determinism: four operational components; model-output determinism: three properties CKS does not require), describes the cell-execution crossing where the boundary is preserved or violated, distinguishes it from four adjacent determinism patterns, enumerates the failure modes that collapse the boundary in either direction, and gives an operational test for whether a system's determinism architecture satisfies it.

1. Why the substrate/model determinism distinction needs to be formalized as standalone

The parent foundational note (A1.10) commits the CKS pattern to a determinism contract: five guarantees the substrate makes, the categories of non-determinism the contract permits, and the anti-patterns that violate it. The integrating-frame note for the contract's decomposition (A2.55) establishes the contract's structure including the substrate/model boundary at the integrating level — naming the boundary alongside the five guarantees, the allowed non-determinism, and regression testing as an integrated whole. This note formalizes the boundary itself as having independent architectural content defensible at the architectural-pattern level, separable from the rest of the contract's components.

The motivating cases are deployments where substrate determinism must be operationally specified in the presence of LLM components whose outputs vary across executions. Without a precise statement of which determinism is required (substrate representation) and which is not (LLM model-output), implementations drift in one of two directions. Some over-engineer by requiring full determinism at the LLM layer — temperature=0, fixed seeds, deterministic-sampling overrides — producing fragile architectures whose viability depends on specific LLM behaviors that are not portable across vendors and may not survive model upgrades. Others under-engineer by allowing non-determinism throughout, including in substrate representation, breaking path retraceability per A1.07, source-of-truth per A1.08, and the AI-as-substrate-mediator commitment per A1.04. The standalone treatment makes the boundary specific and defends against both drifts.

The distinction is load-bearing for several downstream commitments. A1.04 commits the LLM to operate as substrate mediator with non-determinism bounded to the non-substrate layer; the substrate/model distinction is what specifies that bounding architecturally. A1.07 depends on substrate content being deterministic across reads and changes being addressable. A1.08 depends on substrate's representation being deterministic so that authority can be unambiguously assigned. A1.05 depends on the architecture not requiring model-output determinism, so that deployments may use any LLM technology meeting the three minimal requirements. None of these commitments is satisfiable if the distinction collapses.

2. Representation determinism, defined precisely

A substrate satisfies **representation determinism** when its content has a deterministic representation across reads, persists deterministically across operations, changes deterministically through provenance-tracked writes, and composes deterministically as state. The architectural commitment has four operational components.

(a) Deterministic representation across reads. Substrate content has a representation consistent across reads. When substrate state S contains content C, every read against S returns content equivalent to C — same identifiers, relationships, provenance, field values — across cells, readers, and times. This is Guarantee A per A2.57 read at the operational level.

(b) Deterministic persistence across operations. Substrate content persists with its representation across operations that do not modify it. Reads, queries, and derivative-view generations do not alter content; only writes per A2.10 do, and writes are themselves deterministic per (c). This is a representational commitment, not a transactional-durability one — the substrate carries content

unchanged across non-modifying operations, with each subsequent read returning what the most recent write committed.

(c) Deterministic change through provenance-tracked writes. Changes occur as writes per A2.10 with the six provenance fields per A2.40. Each write is atomic with content; the substrate's transition from pre-write state to post-write state is uniquely determined by the content and the provenance. Two writes with identical content and identical provenance produce identical state changes. This grounds Guarantee C per A2.59 — substrate changes are addressable because they are deterministic.

(d) Deterministic state composition. Given substrate state S and a write W with provenance P, the resulting state S′ is uniquely determined by the triple (S, W, P). Order of operations matters only when operations conflict; non-conflicting operations commute deterministically. This is the foundation Guarantee B per A2.58 builds on — cell behavior is rule-equivalent under same substrate state and same orchestration rules because substrate state composes deterministically beneath the cells.

The four components are jointly necessary. A system whose substrate satisfies all four has the substrate-side of the determinism contract; a system that fails any one fails the architectural commitment regardless of how robustly the others hold.

3. Model-output determinism — defined precisely, and why CKS does not require it

Model-output determinism is the property that LLM outputs are bit-identical across identical-input executions. As an architectural commitment it would have three components, none of which CKS requires.

(a) Token-level identity. The LLM produces the same sequence of tokens for the same input prompt across executions.

(b) Probability-distribution identity. The LLM's probability distributions over tokens at each generation step are identical across executions, with sampling either deterministic given a fixed seed or absent (e.g., greedy decoding).

(c) Internal-reasoning-path identity. The LLM's internal computation — attention patterns, layer activations, intermediate hidden states — is identical across executions for identical inputs.

CKS does not require any of (a)–(c). LLM token outputs vary across executions due to sampling, temperature, batching effects, hardware non-determinism, and other model-internal sources of variation; this variation is architecturally allowed per A2.62 category (a). The architectural commitment is that LLM non-determinism is **bounded** — it does not propagate from the LLM layer into substrate representation — not that LLM outputs are deterministic.

Three reasons foreclose making model-output determinism a CKS commitment. **Operational infeasibility:** current LLM technology does not reliably produce bit-identical outputs across identical-input executions even under deterministic-sampling configurations; across vendors, model versions, hardware substrates, and batching regimes, identical inputs produce non-identical outputs in ways outside the architecture's control. **Architectural orthogonality:** model-output determinism is a property of the LLM layer, not the substrate layer; whether the LLM exhibits deterministic behavior is orthogonal to the architecture's substrate-determinism commitment. **Tool-agnosticism preservation:** per A1.05 and A2.24–A2.26, deployments use various LLM technologies with various

non-determinism characteristics, and requiring LLM determinism would constrain the LLM choice, narrowing the architecture's scope in a way the tool-agnosticism commitment forbids.

The combined effect: representation determinism is required of substrate; model-output determinism is neither required of LLM nor architecturally prohibited at the LLM layer. The architecture is silent on the LLM's determinism; it is only specific about the boundary at which LLM non-determinism stops.

4. The substrate/model boundary at cell-execution

The substrate/model boundary is preserved or violated at one operational locus: cell-execution per A2.10. A cell crosses the boundary in four steps, each carrying a determinism property the architecture commits to.

(a) Read crossing. The cell reads from substrate per Property A (A2.19). The read is determined by Guarantee A — the cell receives substrate content with deterministic representation, identical across cells reading the same substrate state. Whatever non-determinism the cell experiences downstream, the content as the cell receives it is deterministic.

(b) LLM execution. The LLM produces outputs that may vary across executions (model-output non-determinism per §3). LLM outputs are LLM-layer content, not substrate-layer content; the substrate carries no record of LLM intermediate outputs unless an orchestration rule explicitly writes them. Variation here is the category (a) non-determinism per A2.62, to which the substrate/model boundary is the bound.

(c) Cell-write structuring. The cell writes to substrate under orchestration rules per Property B (A2.20). The rule specifies which substrate fields are written, what shape the written content takes, what provenance is recorded, and what conditions the write is conditional on. Rule-governance bounds the propagation of LLM non-determinism: what gets written is rule-conformant. Two cell executions over identical substrate state and identical rules may produce variation in written content, but the variation is bounded to what the rule structure allows.

(d) Substrate write commit. The substrate commits the write per A2.10 with provenance per A2.40. The substrate's representation of the written content is deterministic per Guarantee A — once committed, the content is what reads return, identically across readers and times. Provenance is itself deterministic representation.

The boundary is preserved when each of (a)–(d) holds. (b) is where non-determinism is allowed; (a), (c), and (d) are where the boundary preserves substrate's commitments against LLM-layer variation. Stated compactly: LLM non-determinism is irrelevant to the substrate's representation, persistence, and read behavior. LLM-layer variation in the path between read and write is architecturally allowed; substrate-layer variation is architecturally forbidden.

5. What the distinction is not — limits of the standalone treatment and adjacent determinism patterns

Stating precisely what the distinction does not claim, and distinguishing it from four adjacent determinism patterns commonly conflated with it, prevents the standalone framing from being read as either weaker or stronger than the architectural commitment supports.

Limits of the standalone treatment. The distinction does not claim that representation determinism is operationally trivial to implement; implementation choices (transactional storage, immutable-data architectures, version control, output validation, schema enforcement) are deployment concerns. It does not claim model-output non-determinism is desirable; the architecture is compatible with deterministic and non-deterministic LLM behavior alike. It does not claim the boundary is observable without instrumentation; verifying it requires regression testing per A2.63 and provenance review per A2.40. It does not foreclose future LLM technology being deterministic; if such technology emerges, model-output determinism would be a property the LLM layer happens to have, not one the architecture requires.

Not transactional consistency. Database transaction properties (atomicity, consistency, isolation, durability) describe how concurrent transactions interact and how committed state survives failures. Representation determinism commits substrate content's representational consistency across reads, not transaction-concurrency semantics. A system with strong transactional consistency may fail representation determinism; a system with strong representation determinism may rely on weaker transactional consistency provided substrate-internal determinism per Guarantee E (A2.61) holds.

Not deterministic execution semantics. Reproducible-computation guarantees — sought in deterministic-build systems and reproducible-research toolchains — require program execution to produce bit-identical outputs across identical-input executions. Representation determinism commits the substrate's representation, not the computation that operates over it. A system with deterministic execution semantics may fail representation determinism; a system with representation determinism may rely on non-deterministic execution at the LLM layer (per A2.62 category (a)) provided substrate representation per Guarantee A holds.

Not deterministic algorithmic behavior. Algorithm-level reproducibility operates at the algorithmic layer; representation determinism operates at the substrate-content layer. Algorithms running over the substrate (search, ranking, summarization, derived-view generation) may be reproducible or non-reproducible; substrate's representation determinism is independent of their reproducibility. Cell behavior under Guarantee B is rule-equivalent, not algorithmically bit-identical, and the substrate representation grounding the cell's reads is deterministic regardless.

Not idempotent operations. Idempotency is a property of specific operations. Representation determinism is about substrate content's representation across reads and persistence, not about whether operations are repeatable safely. A non-idempotent operation that records its own provenance per A2.40 is fully compatible with the architecture; an idempotent operation that produces non-deterministic substrate representation violates the architecture regardless of how safely it can be re-applied.

The four adjacencies share surface features with representation determinism but each addresses a different architectural concern. The substrate/model distinction is narrower in scope than each and reducible to none.

6. Failure modes that violate the distinction

Eight failure modes recur in implementations that approximate CKS without preserving the substrate/model determinism boundary. Each names a way the boundary collapses, in either direction.

(a) Over-claim: requiring LLM determinism. The implementation requires LLM outputs to be bit-identical across identical-input executions, configuring the LLM with temperature=0, fixed seeds, or deterministic-sampling overrides. The architecture is constrained to specific LLM configurations and may not survive model upgrades; tool-agnosticism per A1.05 fails.

(b) Under-claim: non-determinism in substrate representation. Substrate content varies across reads — different cells reading the same substrate state receive different content. Guarantee A per A2.57 fails; downstream commitments fragment.

(c) Boundary collapse at cell-execution. The implementation fails to structure cell-write under orchestration rules per Property B; LLM outputs are written directly to substrate as unstructured text. The cell-execution boundary per §4 fails at step (c); LLM non-determinism propagates to substrate as unbounded variation.

(d) Provenance non-determinism. Provenance per A2.40 varies across writes — timestamps that drift, writer attribution that rotates, rule references that resolve differently across executions. Guarantee C per A2.59 fails; path retraceability per A1.07 breaks at the provenance layer.

(e) Read-time content variation. Substrate content is transformed at read time in non-deterministic ways — format conversions that vary by reader configuration, derivations that vary by execution context, language-aware reformatting that differs across reads. Reads return varying content for identical substrate state; Guarantee A fails operationally.

(f) Eventual-consistency reads without read repair. The implementation operates in an eventually-consistent storage architecture without read repair, producing reads that vary depending on which replica responds. Representation determinism per Guarantee A fails operationally even where eventual-consistency convergence is correct, because the boundary's commitment is consistency-across-reads, not consistency-across-time.

(g) LLM-output-as-substrate-content. LLM outputs are committed to substrate directly, without rule-structured writes per Property B. The substrate carries non-deterministic content as authoritative; the boundary at cell-execution fails at step (c).

(h) Determinism-feature-flag. Substrate determinism is treated as a deployment-configurable option, with deployments choosing whether substrate behaves deterministically. Determinism becomes a tunable feature, not an architectural commitment; the boundary is decoupled from the architecture and rebuilt as a deployment concern.

In each failure mode the system may continue operating usefully for some purpose. What it loses is the architectural property the substrate/model determinism distinction names. A system exhibiting any of (a)–(h) does not preserve the boundary in the architectural sense, regardless of how reliably it appears to behave in any given session.

7. Operational test

A system satisfies the substrate/model determinism distinction if and only if all of the following are true at all times during the substrate's existence:

1. **Deterministic representation across reads.** Two reads of identical substrate state, by any reader through any cell that supports reading, return identical content (per Guarantee A per A2.57).

2. **Deterministic persistence.** Substrate content unchanged by any write per A2.10 persists with its representation unchanged across operations.

3. **Deterministic provenance-tracked change.** Writes per A2.10 with provenance per A2.40 produce uniquely-determined post-write state given the pre-write state, the content, and the provenance.

4. **Bounded LLM non-determinism.** At the cell-execution boundary per §4, cell-writes are structured under orchestration rules per Property B (A2.20), with rule-governance bounding the variation that LLM non-determinism may produce.

5. **No requirement of LLM model-output determinism.** Deployment is operationally compatible with LLM technologies regardless of their determinism characteristics (per A1.05).

6. **Operational testability of the boundary.** Substrate determinism is verifiable through regression testing per A2.63; LLM non-determinism's bounding to the non-substrate layer is verifiable through provenance review per A2.40.

A system that fails any of (1)–(6) does not satisfy the substrate/model determinism distinction in the architectural sense, even if it appears to operate with some determinism properties. The treatment here names the invariants any test must verify and leaves the test mechanism open, in keeping with the operational tests in A1.10 and the human-governed note.

———

The substrate/model determinism distinction is the load-bearing architectural boundary that the determinism contract operates on. Implementations under pressure to deliver reliable AI behavior consistently drift toward either over-claiming determinism (requiring LLM determinism, producing fragile architectures) or under-claiming (allowing non-determinism throughout, breaking retraceability and source-of-truth). The drift is steady because the boundary is operationally subtle and rhetorically harder to explain than either extreme. Naming it as a standalone architectural commitment — with representation determinism specified by the four components in §2, model-output determinism specified as not-required by the three properties in §3, the cell-execution boundary specified by the four crossings in §4, the limits and adjacent patterns distinguished in §5, the eight failure modes enumerated in §6, and the operational test in §7 — gives downstream readers a precise specification of what determinism the architecture requires and what it explicitly does not. Subsequent specialization notes A2.57–A2.63 elaborate each guarantee, the allowed non-determinism categories, and regression testing; together they give the full operational decomposition of A1.10.

———

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Representation Determinism vs. Model-Output Determinism: The Standalone Architectural Distinction Between What CKS Requires Deterministic and What It Does Not.* May 5, 2026. ORCID: 0009-0004-8065-3235.